

西安交通大學

XI'AN JIAOTONG UNIVERSITY

基于 GPU 的服务资源卸载技术探索

— 透明存储压缩卸载系统 GPUC

2026 全国大学生计算机系统能力大赛 — OS 功能挑战赛道

西安交通大学 · 队伍 **404notfound**

为什么需要 GPU 存储压缩卸载?

大数据、日志、数据库、编译缓存等场景中，**磁盘 I/O 是瓶颈**。存储压缩可减少磁盘占用、提升有效读带宽，但传统方案存在局限：

方案	问题	侵入性
文件系统内置压缩	btrfs/zstd 等，需格式化迁移	**高**
应用层压缩	需修改业务代码，无法通用	**高**
FUSE / device-mapper	部署复杂或风险高	**中**
内核模块	需编译加载，稳定性风险	**高**

关键问题：如何在**不修改内核、文件系统、应用**的前提下，透明地把存储压缩/解压卸载到 GPU?

GPU 具有高并行、高内存带宽，适合解压任务卸载 — 这是本项目的出发点。

核心思路 — 非侵入式透明 GPU 压缩卸载

通过 **LD_PRELOAD** 用户态拦截层实现透明 GPU 压缩/解压，无需修改内核、文件系统或应用程序。

libgpuhook.so
LD_PRELOAD 拦截层
拦截 25+ POSIX 系统调用

→

gpu_compress_daemon
离线 GPU 压缩守护进程
后台异步压缩文件

→

ext4 / xfs
底层文件系统
完全无需改动

三条核心路径

写路径：应用 write() → Hook 附加 RAW header → 物理写入 → Daemon 后台 GPU 压缩

读路径：应用 read() → Hook 读取 header → 查 block table → GPU batch 解压 → 返回原始数据

mmap 路径：应用 mmap() → 分配匿名内存 → 同步预加载 16 MB + 后台异步解压 → 立即返回地址

核心优势：应用完全无感知，无需修改任何代码；GPU 失败时自动 fallback 到 CPU；支持可插拔压缩后端（SGLZ / LZ4）

GPUC 文件格式

自描述的 GPUC 压缩文件格式，Magic = **0x47505543** ("GPUC")，block_size 固定 128 KB。

文件结构

区域	大小	说明
Header	**56 B**	magic、version、orig_size、state、comp_algo、header_size、mtime、ctime
Block Index Table	**num_blocks × 24 B**	每 block 的物理偏移、压缩后大小、CRC32 校验值
压缩数据	变长	紧凑格式存储，稀疏文件空洞保持 128 KB 逻辑对齐

状态机

RAW（初始写入）→ Daemon 后台压缩 → **COMPRESSED / INLINE**（小文件内联）

· 低可压缩率文件保持 RAW 状态，避免压缩开销

· Daemon 使用 **rename()** 原子替换，保证文件始终可用

· 压缩后不足 128 KB 的块用 lseek 留空洞，保持逻辑对齐

系统架构与三条核心路径

应用层：fio, cp, dd, SQLite, LevelDB, 数据库 ...

↓

LD_PRELOAD Hook (libgpuhook.so)

open / write / close ｜ read / pread64 → RAW 透传 / COMPRESSED 自动解压 ｜ mmap / munmap / msync ｜ fstat → 报告逻辑大小

↓

Offline Daemon (gpu_compress_daemon)

SGLZ（自定义 CUDA） ｜ LZ4（NVIDIA nvCOMP，可选） ｜ atomic rename 替换

↓

底层文件系统：ext4 / xfs（无需改动）

写路径：应用 write() → Hook 附加 RAW header → 物理写入 → Daemon 后台压缩 → COMPRESSED

读路径：应用 read() → Hook 读取 header → 查 block table → GPU batch 解压 → 返回原始数据

mmap 路径：应用 mmap() → 分配匿名内存 → 同步预加载 16 MB + 后台异步解压 → 立即返回地址

自研 SGLZ 压缩算法

算法设计

· 128 KB block = **32 × 4 KB chunks**

· 每 chunk 独立 LZ77 压缩/解压

· **3-byte hash**，256 槽位

· 滑动窗口 **2048 B**

· 不可压缩 chunk 用 **0xFFFF** 标记 fallback

· 不依赖闭源 nvCOMP

· 体现 GPU 通用算力

· 对高重复数据压缩率极好

压缩效果

数据类型	SGLZ	LZ4
全零数据	**3.2%**	**3.2%**
可压缩重复模式	**3.2%**	—

核心优势

· 完全自主可控，不依赖第三方闭源库

· 高重复数据压缩率极低（3.2%）

· 自动检测不可压缩数据并 fallback

· 与 nvCOMP LZ4 后端共存，可插拔切换

可插拔后端 — nvCOMP LZ4

切换方式

```
GPUC_ALGO=lz4 ./bin/gpu_compress_daemon <dir>
```

效果对比（Linux headers，~35.7 MB）

方案	压缩率	128k 顺序读带宽	说明
GPUC (SGLZ)	82.5%	279 MB/s	自研算法，通用算力
GPUC (LZ4)	**38.6%**	**581 MB/s**	**nvCOMP 4.2.0.11**

结论

· 文本类数据压缩率提升 **53.2%**（相对 SGLZ）

· 读带宽同时翻倍，实现"压缩率更高、读带宽更高"的双赢

· Hook 读取时根据 comp_algo 自动选择 SGLZ / LZ4 解压路径

· CPU fallback 基于 liblz4，GPU 不可用时仍可正常解压

核心优化体系

优化项	解决问题	效果
16 CUDA Stream + TLS	32 线程竞争单 Stream	**消除同步瓶颈**
Device Memory Pool	逐块 cudaMalloc/Free	**零动态分配开销**
io_uring 批量 pread	syscall 开销高	**减少系统调用次数**
自适应 Read-Ahead	小 bs 无法形成 batch	**动态窗口 0→4→8→16 blocks**
Pinned Memory	driver 中转 memcpy	**消除冗余拷贝**
Compact Kernel	CPU 侧紧凑格式展开	**消除 CPU memcpy**
mmap 按需解压	全量预解压阻塞 mmap()	**返回时间 2414 ms → 231 ms**

综合效果：从早期 jobs=32 时骤降至 **267 MB/s**，优化后恢复至 Baseline 的 **99.8%**

关键性能数据 — 顺序读

测试条件：fio 顺序读 128k（512 MB，page cache 命中）· 环境：Tesla V100-SXM2-32GB · CUDA 12.8 · ext4

场景	zero	rand	rep
Baseline	450.9 MB/s	448.0 MB/s	451.5 MB/s
Hook RAW	451.7 MB/s	443.0 MB/s	447.2 MB/s
GPU Compressed	**458.3 MB/s**	**433.0 MB/s**	**442.4 MB/s**

结论

· GPU Compressed 与 Baseline **基本持平**

· GPU 解压 overhead 已被完全摊销，读性能无损耗

多线程并发读取（zero, 128KB）

jobs	Baseline	GPU Compressed	扩展比
1	456.3	466.3	1.00x
16	3768.1	**3797.7**	**8.14x**

高并发扩展性 — jobs=32 恢复至 99.8%

问题：早期 jobs=32 时 GPUC 骤降至 **267 MB/s**，经系统优化后全面恢复。

优化后（direct=1）

数据	bs	Baseline (MB/s)	GPUC (MB/s)	比例
zero	128k	4900.2	**4889.1**	**99.8%**
zero	1M	414.3	**412.9**	**99.7%**
rand	128k	396.6	**395.8**	**99.8%**
rand	1M	435.2	**425.2**	**97.7%**

关键优化措施：

· 16 CUDA Stream + 线程本地存储（TLS）消除同步瓶颈

· Device Memory Pool 消除 cudaMalloc/Free 开销

· Pinned Memory + Compact Kernel 消除冗余 CPU memcpy

· io_uring 批量 pread 减少系统调用

mmap 按需解压与压缩效果

mmap 优化效果

文件大小	优化前	优化后	提升
64 MB	420.9 ms	226.3 ms	1.9x
256 MB	1263.3 ms	229.8 ms	5.5x
512 MB	2414.5 ms	**231.4 ms**	**10.4x**

同步 16 MB 预加载 + 后台异步解压，512 MB 文件 mmap 返回时间从 2414 ms 降至 231 ms。

压缩率对比

数据类型	SGLZ	LZ4	说明
全零数据	**3.2%**	**3.2%**	极佳
Linux headers	82.5%	**38.6%**	**LZ4 优势**
混合数据	53.1%	—	中等
随机数据	100%	100%	fallback

低可压缩率文件自动保持 RAW，避免压缩开销。随机数据 100% fallback。

真实 Workload 评估

Workload	关键指标	Hook/Baseline	状态
SQLite	建表 / 插入 / SELECT	—	**PASS**
LevelDB	fillseq / readseq / readrandom	—	**PASS**
ccache cold	冷编译缓存	86.4%	可接受
ccache hot	热编译缓存	79.7%	可接受
ML 小文件加载	大量小文件加载	15–17%	overhead 大

关键修复 — 数据库白名单

· 数据库文件附加 56 B RAW header 会破坏数据库 magic，导致无法识别

· 白名单机制：以下后缀文件不附加 GPUC header，透传处理

.db、.wal、.log、.ldb、.sst、CURRENT、MANIFEST-*

· 修复后 SQLite 和 LevelDB 全部测试通过

生产级指标与稳定性

尾延迟（randread 128k）

指标	GPUC	Baseline
p99	**457 µs**	更高
p999	**578 µs**	更高
IOPS	**~3355**	—

GPUC 尾延迟均低于 Baseline

长期稳定性

- 连续读取 1 GB 压缩文件 **30 分钟**
- 带宽稳定 **159.2 MB/s**
- **零 CRC32 错误**
- 无内存泄漏，无资源耗尽

容错边界测试（5 项全 PASS）

1. Daemon 压缩时应用并发读/写 — 无数据竞争，文件一致性保证
2. GPU 显存不足自动 fallback CPU — 自适应降级，服务不中断
3. 磁盘空间不足正确返回 ENOSPC — 错误码透传，符合 POSIX 语义
4. kill -9 后文件一致性 — temp file + atomic rename，无半完成文件
5. fsync 语义正确 — 数据持久化保证，不丢失用户数据

与现有方案对比

维度	btrfs zstd	应用层压缩	GPUC（本方案）
侵入性	**高**	**高**	**无**
部署难度	中	高	**低**
性能上限	高（内核态）	取决于实现	中（用户态+GPU）
GPU 卸载	否	可选	**是**
适用场景	新建系统	特定应用	**现有系统透明叠加**

结论：

· 内核态方案（btrfs zstd）性能最优但侵入性强，需格式化迁移

· GPUC 以**部署灵活性**为核心价值，适合无法改造底层存储的环境

· 通过 LD_PRELOAD 实现"零侵入"透明叠加，是现有系统的最优解

项目创新点

1. **非侵入式 GPU 存储压缩卸载** — 通过 LD_PRELOAD 实现透明 GPU 压缩/解压，无需改内核/文件系统/应用
2. **自研 SGLZ CUDA 算法** — 不依赖 nvCOMP，体现 GPU 通用算力，高重复数据压缩率达 3.2%
3. **完整性能优化体系** — 16-stream、内存池、io_uring、read-ahead、紧凑 kernel，jobs=32 恢复至 Baseline 99.8%
4. **mmap 按需解压** — 同步 16 MB + 后台异步解压，512 MB 文件 mmap 返回时间从 2414 ms 降至 231 ms
5. **可插拔双后端** — SGLZ 与 nvCOMP LZ4 共存，兼顾自主知识产权与真实数据压缩率
6. **自适应高可用** — GPU 高负载、显存不足、低可压缩率等场景自动降级，保证鲁棒性
7. **完整工程化测试体系** — 17+ 自动化测试脚本覆盖性能、稳定性、容错、真实 workload

团队分工与协作

角色	职责	成员
队长 / 系统架构	整体架构、Hook 与 Daemon 设计、项目整合	**朱嘉乐**
GPU 后端开发	SGLZ CUDA kernel、nvCOMP LZ4 后端集成	朱嘉乐
存储与 I/O 优化	io_uring、内存池、read-ahead、mmap 优化	贾茹
测试与评测	fio 基准、消融实验、稳定性与边界测试	贾茹
文档与答辩材料	README、开发文档、PPT、演示视频	边程曦

队伍：**404notfound** · 西安交通大学 · 2026 全国大学生计算机系统能力大赛

开发时间线

阶段	时间	关键产出
立项与调研	2026-05-05	项目架构方案、README 初稿
核心系统	2026-06-09	LD_PRELOAD Hook + GPU Daemon + SGLZ 压缩
全面评测	2026-06-10	消融实验、Batch Size 扫描、P1/P2 workload
性能优化	2026-06-11	多 CUDA Stream、mmap 按需解压（10.4x）
生产验证	2026-06-12 ~ 06-16	真实负载、DB 白名单、自适应降级、稳定性
最终集成	2026-06-17	集成 NVIDIA nvCOMP LZ4 GPU 后端
文档整理	2026-06-26	补齐设计方案开发文档

git 提交：当前 **20+** 次提交，完整记录迭代过程

合规声明

开源协议

· 源代码许可证：**Apache-2.0**（LICENSE）

· 文档/答辩材料许可证：**CC BY-SA 4.0**（LICENSE-docs）

第三方依赖

· 第三方依赖说明：docs/THIRD_PARTY.md

AI 工具使用声明

· AI 工具使用声明：docs/AI_TOOL_DECLARATION.md

原创声明

· 本项目代码均为团队**原创开发**，未基于任何往届作品或开源项目二次开发

一键复现

```
# 环境检查
bash scripts/check_env.sh
# 编译
make clean && make all
# 运行测试
make test
# 一键 DEMO
bash scripts/demo.sh
# 性能基准
bash tests/benchmark_fio.sh
# LZ4 后端
bash tests/nvcomp_lz4.sh
```

总结与展望

总结

1. 通过 LD_PRELOAD 实现了**非侵入式** GPU 存储压缩卸载

2. 多 CUDA Stream、内存池、io_uring、read-ahead 等优化使读性能达到 **Baseline 水平**

3. nvCOMP LZ4 后端将文本类数据压缩率从 82.5% 降至 **38.6%**，读带宽提升至 **581 MB/s**

4. 具备 CPU fallback、CRC32、原子替换、并发安全等**生产级基础**

未来方向

· CUDA Graph 固化 kernel 启动

· userfaultfd 页级 mmap 按需解压

· 写路径实时 GPU 压缩

· per-fd 多 block LRU cache

感谢大赛组委会、指导老师、开源社区

西安交通大学 · 404notfound · 2026 全国大学生计算机系统能力大赛